SQL SERVER UNTUK QA

Persiapan Data Test yang Mandiri

INSERT • UPDATE • DELETE • BACKUP • MASKING • RANDOM DATA

Agenda

1. **INSERT - Menyisipkan Data Test**
   Teknik insert data dengan berbagai skenario
2. **UPDATE - Modifikasi Data**
   Update data untuk testing state changes
3. **DELETE - Cleanup Data**
   Menghapus data test setelah selesai
4. **BACKUP & RESTORE**
   Save & restore state untuk repeatability
5. **Data MASKING**
   Protect sensitive data di test environment
6. **RANDOM & DUMMY Data**
   Generate data realistis dengan SQL

INSERT: Menyisipkan Data Test

Basic INSERT Syntax

- Single row insert
- Multiple rows insert
- Insert dengan SELECT
- Insert dengan DEFAULT values
- Insert dengan OUTPUT clause

💡 Tips

Gunakan OUTPUT untuk capture ID yang baru di-insert

Contoh SQL

```
-- Single insert
INSERT INTO Users (Email, Name)
VALUES ('test@qa.com', 'Test User');

-- Multiple insert
INSERT INTO Products
  (ProductName, Price, Stock)
VALUES
  ('Product A', 100, 50),
  ('Product B', 200, 30),
  ('Product C', 150, 0);

-- Insert with OUTPUT
INSERT INTO Orders (UserId, Total)
OUTPUT INSERTED.OrderId
VALUES (1, 500);
```

INSERT: Teknik Advanced

INSERT dengan IDENTITY

```
-- Allow manual ID insert
SET IDENTITY_INSERT Users ON;

INSERT INTO Users (UserId, Email)
VALUES (999, 'specific@id.com');

SET IDENTITY_INSERT Users OFF;
```

Bulk INSERT dari File

```
-- Import dari CSV
BULK INSERT Products
FROM 'C:\data\products.csv'
WITH (
  FIELDTERMINATOR = ',',
  ROWTERMINATOR = '\n',
  FIRSTROW = 2
);
```

INSERT dari SELECT

```
-- Copy data dari table lain
INSERT INTO TestOrders
  (UserId, ProductId, Qty)
SELECT UserId, ProductId, Qty
FROM ProdOrders
WHERE CreatedDate > '2024-01-01';
```

UPDATE: Modifikasi Data Test

Kapan Perlu UPDATE?

- Testing state changes (Pending → Approved)
- Setup edge case scenarios (expired dates, zero stock)
- Simulate data corruption atau invalid state
- Update timestamps untuk testing time-based logic

Basic UPDATE

```
-- Update single field
UPDATE Users
SET Status = 'Inactive'
WHERE Email = 'test@qa.com';

-- Update multiple fields
UPDATE Orders
SET Status = 'Cancelled',
    CancelledDate = GETDATE(),
    CancelReason = 'Test'
WHERE OrderId = 12345;
```

UPDATE dengan JOIN

```
-- Update dari table lain
UPDATE o
SET o.TotalAmount = oi.Total
FROM Orders o
INNER JOIN (
  SELECT OrderId,
         SUM(Price*Qty) as Total
  FROM OrderItems
  GROUP BY OrderId
) oi ON o.OrderId = oi.OrderId
WHERE o.Status = 'Draft';
```

DELETE: Cleanup Data Test

⚠️ **PENTING: Selalu gunakan WHERE clause! DELETE tanpa WHERE akan hapus SEMUA data!**

DELETE Basic

```
-- Delete specific rows
DELETE FROM TestUsers
WHERE Email LIKE '%@test.com';

-- Delete with JOIN
DELETE od
FROM OrderDetails od
INNER JOIN Orders o
  ON od.OrderId = o.OrderId
WHERE o.CreatedDate < '2024-01-
01';
```

TRUNCATE (Fast Delete)

```
-- Remove all rows (fast)
TRUNCATE TABLE TestOrders;

-- Cannot use with FK
-- Cannot use WHERE
-- Resets IDENTITY counter

-- Good for:
-- • Cleanup entire table
-- • No FK references
```

DELETE dengan OUTPUT

```
-- Capture deleted data
DELETE FROM TempProducts
OUTPUT DELETED.ProductId,
       DELETED.ProductName
INTO @DeletedItems
WHERE IsActive = 0;

-- Useful for:
-- • Audit log
-- • Rollback capability
```

BACKUP & RESTORE Data

Essential untuk repeatability testing - save state sebelum test, restore setelah selesai

1. Full Database Backup

```
-- Backup entire database
BACKUP DATABASE TestDB
TO DISK = 'C:\Backup\TestDB.bak'
WITH FORMAT, NAME = 'TestDB-Full';
```

2. Table-Level Backup (SELECT INTO)

```
-- Backup specific table
SELECT * INTO Users_Backup
FROM Users;
```

Restore Database

```
-- Restore from backup
RESTORE DATABASE TestDB
FROM DISK = 'C:\Backup\TestDB.bak'
WITH REPLACE;
```

Restore Table

```
-- Restore from backup table
DELETE FROM Users;
INSERT INTO Users
SELECT * FROM Users_Backup;
```

Data MASKING: Protect Sensitive Data

Masking = menyembunyikan data sensitif (email, phone, credit card) saat copy dari production ke test environment

Email Masking

```
UPDATE Users
SET Email =
  CONCAT(
    'user',
    UserId,
    '@test.com'
  )
WHERE Environment = 'Test';
```

Phone Masking

```
UPDATE Customers
SET Phone =
  CONCAT(
    '08XX-XXXX-',
    RIGHT(Phone, 4)
  );

-- Result: 08XX-XXXX-1234
```

Name Masking

```
UPDATE Users
SET FirstName = 'Test',
    LastName =
      CONCAT(
        'User',
        UserId
      );
-- Result: Test User123
```

Credit Card Masking

```
UPDATE PaymentMethods
SET CardNumber =
  CONCAT(
    'XXXX-XXXX-XXXX-',
    RIGHT(CardNumber, 4)
  );
-- Result: XXXX-XXXX-XXXX-5678
```

Address Masking

```
UPDATE Addresses
SET StreetAddress =
    'Test Address',
    City = 'Test City',
    PostalCode = '12345';
```

Dynamic Data Masking

```
-- SQL Server feature
ALTER TABLE Users
ALTER COLUMN Email
  ADD MASKED WITH
  (FUNCTION = 'email()');

-- Auto-mask for non-admin
```

Generate RANDOM Data

Gunakan fungsi SQL Server untuk generate data random yang realistis

SQL Server Random Functions

NEWID()	Generate unique GUID
RAND()	Random float 0-1
ABS(CHECKSUM(NEWID()))	Random integer
GETDATE()	Current datetime
DATEADD()	Date arithmetic

💡 *Combine untuk hasil lebih realistis*

Contoh Random Data

```
-- Random integer (1-100)
ABS(CHECKSUM(NEWID())) % 100 + 1

-- Random price (10.00 - 1000.00)
CAST(RAND() * 990 + 10 AS DECIMAL(10,2))

-- Random date (last 365 days)
DATEADD(DAY,
  -ABS(CHECKSUM(NEWID())) % 365,
  GETDATE())

-- Random email
CONCAT('user', NEWID(), '@test.com')

-- Random status
CASE ABS(CHECKSUM(NEWID())) % 3
  WHEN 0 THEN 'Active'
  WHEN 1 THEN 'Pending'
  ELSE 'Inactive' END
```

Generate DUMMY Data: Complete Example

```
-- Create temp numbers table
WITH Numbers AS
  SELECT TOP 1000 ROW_NUMBER() OVER (ORDER BY (SELECT NULL)) as Num
  FROM sys.all_columns
)
```

Generate 1000 dummy users dengan data random

```
-- Insert dummy users
INSERT INTO Users (
  Email, FirstName, LastName, BirthDate,
  Phone, Status, CreatedDate, Balance
)
SELECT
  -- Random email
  CONCAT('user', Num, '_', LEFT(NEWID(), 8), '@test.com'),

  -- Random first name from list
  CASE ABS(CHECKSUM(NEWID())) % 5
    WHEN 0 THEN 'John' WHEN 1 THEN 'Jane'
    WHEN 2 THEN 'Bob' WHEN 3 THEN 'Alice' ELSE 'Charlie' END,

  -- Last name
  CONCAT('TestUser', Num),

  -- Random birth date (18-80 years ago)
  DATEADD(YEAR, -(ABS(CHECKSUM(NEWID())) % 62 + 18), GETDATE()),

  -- Random phone
  CONCAT('08', ABS(CHECKSUM(NEWID())) % 90000000 + 10000000),

  -- Random status
  CASE ABS(CHECKSUM(NEWID())) % 3
    WHEN 0 THEN 'Active' WHEN 1 THEN 'Pending' ELSE 'Inactive' END,

  -- Random created date (last 365 days)
  DATEADD(DAY, -ABS(CHECKSUM(NEWID())) % 365, GETDATE()),

  -- Random balance (0-10000)
```

TRANSACTION & ROLLBACK

Safety net untuk testing - rollback jika ada error

Basic Transaction Pattern

```
BEGIN TRANSACTION;

-- Your test data setup
INSERT INTO Users (...) VALUES (...);
UPDATE Orders SET ...;

-- Run your tests here

-- Rollback to clean state
ROLLBACK TRANSACTION;

-- OR commit if want to keep
-- COMMIT TRANSACTION;
```

With Error Handling

```
BEGIN TRY
  BEGIN TRANSACTION;

  -- Your operations
  INSERT INTO Users (...);
  UPDATE Orders ...;

  COMMIT TRANSACTION;
  PRINT 'Success!';
END TRY
BEGIN CATCH
  ROLLBACK TRANSACTION;

  PRINT 'Error: ' + ERROR_MESSAGE();
END CATCH;
```

Stored Procedures untuk Test Data

Reusable scripts untuk setup & cleanup data test

Contoh: Setup Test User Procedure

```
CREATE PROCEDURE sp_CreateTestUser
  @UserType VARCHAR(20) = 'Customer',
  @WithOrders BIT = 0
AS
BEGIN
  DECLARE @UserId INT;

  -- Create user
  INSERT INTO Users (Email, Name, UserType, Status)
  VALUES (
    CONCAT('test_', NEWID(), '@qa.com'),
    CONCAT('Test ', @UserType),
    @UserType,
    'Active'
  );

  SET @UserId = SCOPE_IDENTITY();

  -- Optionally create orders
  IF @WithOrders = 1
  BEGIN
    INSERT INTO Orders (UserId, Total, Status)
    VALUES (@UserId, 100, 'Completed');
  END

  RETURN @UserId;
END;
```

Keuntungan

- Reusable
- Konsisten
- Parameterized
- Easy to call
- Version control
- Team sharing

Usage:

```
EXEC sp_CreateTestUser
  @UserType='Admin',
  @WithOrders=1;
```

Best Practices

1. **Gunakan Naming Convention**
   Prefix test data: test_, qa_, dummy_ untuk mudah identifikasi

2. **Script Everything**
   Simpan semua SQL scripts di version control (Git)

3. **Isolasi Environment**
   JANGAN pernah test di production database!

4. **Always Use WHERE**
   DELETE/UPDATE tanpa WHERE = disaster!

5. **Backup Before Test**
   Selalu backup sebelum run test data script

6. **Cleanup After Test**
   Delete atau truncate test data setelah selesai

7. **Use Transactions**
   Wrap dalam transaction untuk easy rollback

8. **Document Your Data**
   Comment SQL scripts dengan purpose dan requirement

Common Mistakes & Solutions

❌ **DELETE/UPDATE tanpa WHERE**	*Hapus/ubah SEMUA data!*	✅ Selalu test dengan SELECT dulu, tambah WHERE
❌ **Test di Production DB**	*Corrupt real customer data*	✅ Gunakan dedicated test database
❌ **Hardcode IDs**	*Script gagal saat ID berubah*	✅ Use SELECT untuk dapat ID dinamis
❌ **Lupa Foreign Key**	*Insert gagal, FK violation*	✅ Insert parent records dulu
❌ **No Backup**	*Tidak bisa rollback jika error*	✅ Backup before, restore after

Useful Queries untuk QA

Check Table Size

```
SELECT t.name,
  SUM(p.rows) as RowCount
FROM sys.tables t
JOIN sys.partitions p
  ON t.object_id = p.object_id
GROUP BY t.name;
```

Find Tables with Test Data

```
SELECT DISTINCT
  t.name as TableName
FROM sys.tables t
WHERE EXISTS (
  SELECT 1
  FROM sys.columns c
  WHERE c.object_id = t.object_id
  AND c.name LIKE '%test%'
);
```

Check Foreign Keys

```
SELECT
  fk.name as FKName,
  tp.name as ParentTable,
  tr.name as RefTable
FROM sys.foreign_keys fk
JOIN sys.tables tp
  ON fk.parent_object_id=tp.object_id
JOIN sys.tables tr
  ON
fk.referenced_object_id=tr.object_id;
```

Get Last Inserted ID

```
-- Method 1
SELECT SCOPE_IDENTITY();

-- Method 2
SELECT @@IDENTITY;

-- Method 3 (recommended)
INSERT INTO Users (...)
OUTPUT INSERTED.UserId;
```

Check DB Size

```
EXEC sp_spaceused;

-- OR
SELECT
  name,
  size * 8 / 1024 as SizeMB
FROM sys.database_files;
```

Find Duplicate Data

```
SELECT Email, COUNT(*)
FROM Users
GROUP BY Email
HAVING COUNT(*) > 1;
```

Tools & Resources

SQL Server Management Studio (SSMS)

Primary tool untuk manage SQL Server

Azure Data Studio

Cross-platform alternative untuk SSMS

SQL Server Profiler

Monitor & trace SQL queries

Redgate SQL Data Generator

Generate realistic test data

Mockaroo

Online tool generate test data (CSV import)

Git

Version control untuk SQL scripts

Workflow: Setup Test Data

1	**Backup Database**	Save current state
2	**BEGIN TRANSACTION**	Start safe mode
3	**INSERT Test Data**	Create required records
4	**UPDATE States**	Set specific scenarios
5	**Run Your Tests**	Execute test cases
6	**ROLLBACK/Cleanup**	Return to clean state
7	**Verify Clean**	Check no test data remains

Quick Reference

`INSERT INTO table VALUES (...)`	Add new data
`UPDATE table SET x=y WHERE ...`	Modify data
`DELETE FROM table WHERE ...`	Remove data
`TRUNCATE TABLE table`	Fast delete all
`BACKUP DATABASE db TO DISK='...'`	Backup DB
`SELECT * INTO table_bak FROM table`	Backup table
`BEGIN TRANSACTION / ROLLBACK`	Safe testing
`NEWID()`	Random GUID
`ABS(CHECKSUM(NEWID())) % N`	Random number
`OUTPUT INSERTED.Id`	Get new ID

KEY TAKEAWAYS

- SQL Server adalah tool powerful untuk QA mandiri
- Master INSERT, UPDATE, DELETE, BACKUP/RESTORE
- Gunakan masking untuk protect sensitive data
- Generate random/dummy data dengan fungsi SQL
- Selalu backup & gunakan transactions!
- Document dan version control your scripts

🚀 *Start practicing dengan test database hari ini!*

Questions?

Terima kasih!